

완료 보고서

조강혁

2026.05.19~05.27

내용

1. 개요 (Overview).....	3
1.1 설계 목표	3
1.2 설계 환경 및 사양.....	3
2. 프로젝트 관리 (Project Management)	3
2.1 일정 계획 (Schedule).....	3
3. 이론 및 배경 설명	4
3.1 HW/SW Interface 소개	4
3.1.1 Instruction Set Architecture(ISA).....	4
3.1.2 Memory Map & Application Binary Interface(ABI).....	7
4. RISC-V 32I 아키텍처(Architecture)	9
4.1 Microarchitecture 설계	9
4.1.1 RISC-V CPU Block Diagram.....	9
4.2 Single-Cycle 프로세서	15
5. C 코드 검증 시나리오	20
5.1 Bubble Sort.....	20
5.1.1 main 함수	21
5.1.2 sort 함수	24
5.1.3 swap 함수 분석을 통한 포인터 역참조 및 데이터 교환.....	25
5. Trouble-shooting	27
5.1 Data Memory.....	27
5.1.1 Issue 소개	27

5.1.2 Root Cause 및 트리블 슈팅	27
5.1.3 개선 결과 및 교훈	28
5.2 Negative Slack Issue.....	29
5.2.1 Issue 소개	29
5.1.2 Root Cause 및 트리블 슈팅	29
5.1.3 개선 결과 및 교훈	30
6. 결론	31
6.1 결과 요약.....	31
6.2 배운점 및 고찰.....	31

1. 개요 (Overview)

1.1 설계 목표

Reduced Instruction Set Computer(RISC) 및 하버드 구조로 구성된 RISC-V 32I CPU를 SystemVerilog로 설계하고, C 코드로 작성된 검증 시나리오를 실행하여 설계의 오류 여부를 검증한다.

1.2 설계 환경 및 사양

- EDA Tool: Xilinx 사의 Vivado 프로그램을 사용
- C Compiler: RISC-V(32-bits) gcc 16.1.0를 활용하여 Machine Langague로 변환

2. 프로젝트 관리 (Project Management)

2.1 일정 계획 (Schedule)

2026년 5월 19일부터 27일까지 총 9일간 RISC_V를 이용한 CPU 설계 프로젝트를 진행하였다.

단계	기간	주요 작업
아키텍처 설계	5월 19일	Spec 정의, Datapath Flow 및 Block Diagram 기획
RTL 설계	5월 20-21일	Module 구현 (Register File, ALU, Control Unit, Program Counter 등)
Integration 및 간이검증	5월 22-23일	Top-level 통합 및 Insturction Memory를 이용한 단일 명령어 검증
프로그램 단위 검증	5월 24-25일	어셈블리 코드 기획 및 통합 테스트 완료
프로젝트 완료	5월 26-27일	프로젝트 발표 및 최종 보고서 작성

3. 이론 및 배경 설명

3.1 HW/SW Interface 소개

3.1.1 Instruction Set Architecture(ISA)

RISC-V 32i는 명령어의 디코딩 복잡도를 최소화하기 위해 소스 레지스터 (rs1, rs2)와 목적 레지스터 (rd)의 위치를 고정하고, 명령어 타입(R, S, B, I, U, J)에 따라 Immediate 즉시값을 규칙적으로 배열했다는 특징이 있다. 타입별 32비트의 ISA는 다음과 같다.

a. R-Type (Register)

- 핵심 역할: 프로세서 내부의 두 레지스터 rs1, rs2에 저장된 데이터를 소스로 하여 산술, 논리, 시프트 연산을 수행하고 그 결과값을 레지스터 rd에 저장한다.
- 구조적 특징: 연산의 종류가 많기 때문에 동일한 opcode (0110011) 내에서 funct3와 funct7의 조합을 통해 ADD, SUB, SLL, SRL 등의 구체적인 명령어를 구별합니다. (예: ADD와 SUB는 funct7의 6번째 비트로 구분)

R-TYPE		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
		funct7							rs2				rs1				funct3			rd				opcode									
ADD	rd = rs1 + rs2	0	0	0	0	0	0	0	rs2				rs1				0	0	0	rd				0	1	1	0	0	1	1			
SUB	rd = rs1 - rs2	0	1	0	0	0	0	0	rs2				rs1				0	0	0	rd				0	1	1	0	0	1	1			
SLL	rd = rs1 << rs2	0	0	0	0	0	0	0	rs2				rs1				0	0	1	rd				0	1	1	0	0	1	1			
SLT	rd = (rs1 < rs2)?1:0	0	0	0	0	0	0	0	rs2				rs1				0	1	0	rd				0	1	1	0	0	1	1			
SLTU	rd = (rs1 < rs2)?1:0	0	0	0	0	0	0	0	rs2				rs1				0	1	1	rd				0	1	1	0	0	1	1			
XOR	rd = rs1 ^ rs2	0	0	0	0	0	0	0	rs2				rs1				1	0	0	rd				0	1	1	0	0	1	1			
SRL	rd = rs1 >> rs2	0	0	0	0	0	0	0	rs2				rs1				1	0	1	rd				0	1	1	0	0	1	1			
SRA	rd = rs1 >> rs2	0	1	0	0	0	0	0	rs2				rs1				1	0	1	rd				0	1	1	0	0	1	1			
OR	rd = rs1 rs2	0	0	0	0	0	0	0	rs2				rs1				1	1	0	rd				0	1	1	0	0	1	1			
AND	rd = rs1 & rs2	0	0	0	0	0	0	0	rs2				rs1				1	1	1	rd				0	1	1	0	0	1	1			

b. S-Type (Store)

- 핵심 역할: 레지스터 rs2에 있는 데이터를 메모리 [rs1+imm]에 저장한다.
- 구조적 특징: RISC-V 설계 철학에 따라 소스 레지스터 rs1, rs2 위치를 R-TYPE과 동일하게 유지하면서, 12비트의 Immediate 즉시값을 상위 7비트, 하위 5비트로 쪼개어 배치하는 구조를 가진다.

S-TYPE		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
		imm							rs2				rs1				funct3			imm				opcode									
SB	M[rs1+imm][0:7] = rs2[0:7]	imm[11:5]							rs2				rs1				0	0	0	imm[4:0]				0	1	0	0	0	1	1			
SH	M[rs1+imm][0:15] = rs2[0:15]	imm[11:5]							rs2				rs1				0	0	1	imm[4:0]				0	1	0	0	0	1	1			
SW	M[rs1+imm][0:31] = rs2[0:31]	imm[11:5]							rs2				rs1				0	1	0	imm[4:0]				0	1	0	0	0	1	1			

c. B-Type (Branch)

- **핵심 역할:** 두 레지스터 값 rs1, rs2를 비교하여 조건이 만족할 경우, 현재 PC 값을 기준으로 상대주소(PC+imm) 점프를 수행한다.
- **구조적 특징:** S-Type의 변형으로, 최하위 비트인 imm[0]을 항상 0으로 간주하여 명령어 필드 내에 저장하지 않으면서, MSB인 imm[12]를 최상위 비트에 고정하고 imm[11]을 기존 imm[0] 위치에 대신 넣어줌으로써, 하드웨어 효율을 높이고 Immediate 즉시값 로직의 배선을 단순화하는 전략을 사용하였다.

B-TYPE		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
		imm								rs2			rs1			funct3			imm				opcode										
BEQ	if(rs1 == rs2) PC += imm	imm[12,10:5]								rs2			rs1			0 0 0			imm[4:1,11]				1 1 0 0 0 0 1 1										
BNE	if(rs1 != rs2) PC += imm	imm[12,10:5]								rs2			rs1			0 0 1			imm[4:1,11]				1 1 0 0 0 0 1 1										
BLT	if(rs1 < rs2) PC += imm	imm[12,10:5]								rs2			rs1			1 0 0			imm[4:1,11]				1 1 0 0 0 0 1 1										
BGE	if(rs1 >= rs2) PC += imm	imm[12,10:5]								rs2			rs1			1 0 1			imm[4:1,11]				1 1 0 0 0 0 1 1										
BLTU	if(rs1 < rs2) PC += imm	imm[12,10:5]								rs2			rs1			1 1 0			imm[4:1,11]				1 1 0 0 0 0 1 1										
BGEU	if(rs1 >= rs2) PC += imm	imm[12,10:5]								rs2			rs1			1 1 1			imm[4:1,11]				1 1 0 0 0 0 1 1										

d. I, IL, JL-Type (Immediate)

- **핵심 역할:** 레지스터 값과 명령어 내에 포함된 12비트 Immediate 값을 이용하여 연산, 로드, 또는 간접 점프 시에 사용된다.
- **구조적 특징:** 상위 12비트[31:20]을 하나의 연장된 imm[11:0] 필드로 사용하여 정수형 상수를 표현한다.
 - **I-Type (산술/논리):** ADDI 등 레지스터와 상수의 연산을 수행하고, SLLI, SRLI, SRAI 시프트 연산의 경우 상위 7비트를 funct7처럼 활용하면서 하위 5비트를 시프트양 (shift amount)으로 사용한다.
 - **IL-Type (Load):** LB, LH, LW 등 메모리 주소 [rs1 + imm]에서 데이터를 읽어 레지스터 rd에 저장한다.
 - **JL-Type (JALR):** 레지스터 rs1에 12비트 변위를 더해 대상 주소로 jump하고, 돌아올 주소 PC+4를 레지스터 rd에 저장한다.

I-TYPE		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
		imm												rs1			funct3			rd			opcode										
ADDI	rd = rs1 + imm	imm[11:0]												rs1			0 0 0			rd			0 0 1 0 0 0 1 1										
SLTI	rd = (rs1 < imm)?1:0	imm[11:0]												rs1			0 1 0			rd			0 0 1 0 0 0 1 1										
SLTIU	rd = (rs1 < imm)?1:0	imm[11:0]												rs1			0 1 1			rd			0 0 1 0 0 0 1 1										
XORI	rd = rs1 ^ imm	imm[11:0]												rs1			1 0 0			rd			0 0 1 0 0 0 1 1										
ORI	rd = rs1 imm	imm[11:0]												rs1			1 1 0			rd			0 0 1 0 0 0 1 1										
ANDI	rd = rs1 & imm	imm[11:0]												rs1			1 1 1			rd			0 0 1 0 0 0 1 1										
SLLI	rd = rs1 << imm[0:4]	0	0	0	0	0	0	0	0	shamt				rs1			0 0 1			rd			0 0 1 0 0 0 1 1										
SRLI	rd = rs1 >> imm[0:4]	0	0	0	0	0	0	0	0	shamt				rs1			1 0 1			rd			0 0 1 0 0 0 1 1										
SRAI	rd = rs1 >> imm[0:4]	0	1	0	0	0	0	0	0	shamt				rs1			1 0 1			rd			0 0 1 0 0 0 1 1										

IL-TYPE		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
		imm												rs1			funct3			rd			opcode										
LB	rd = M[rs1+imm][0:7]	imm[11:0]												rs1			0 0 0			rd			0 0 0 0 0 0 1 1										
LH	rd = M[rs1+imm][0:15]	imm[11:0]												rs1			0 0 1			rd			0 0 0 0 0 0 1 1										
LW	rd = M[rs1+imm][0:31]	imm[11:0]												rs1			0 1 0			rd			0 0 0 0 0 0 1 1										
LBU	rd = M[rs1+imm][0:7]	imm[11:0]												rs1			1 0 0			rd			0 0 0 0 0 0 1 1										
LHU	rd = M[rs1+imm][0:15]	imm[11:0]												rs1			1 0 1			rd			0 0 0 0 0 0 1 1										
JL-TYPE		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
		imm												rs1			funct3			rd			opcode										
JALR	rd = PC+4; PC = rs1 + imm	imm[11:0]												rs1			0 0 0			rd			1 1 0 0 0 1 1 1										

e. J-Type (Jump)

- **핵심 역할:** 조건 없이 먼 거리의 주소 (imm)로 무조건 점프를 수행하며, 함수 호출 시 복귀 주소를 저장하는데 사용된다.
- **구조적 특징:** B-Type과 마찬가지로 하드웨어 로직 최적화 및 효율을 위해 즉시값은 비트 순서를 뒤섞인 스캔블링 구조 (imm[20, 10:1, 11, 19:12])를 가지며, 최하위 비트 imm[0]은 0으로 간주되어 생략된다. 점프 후 돌아올 주소 PC+4를 레지스터 rd에 저장한다.

J-TYPE		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
		imm																		rd			opcode										
JAL	rd = PC+4; PC += imm	imm[20,10:1,11,19:12]																		rd			1 1 0 1 1 1 1										

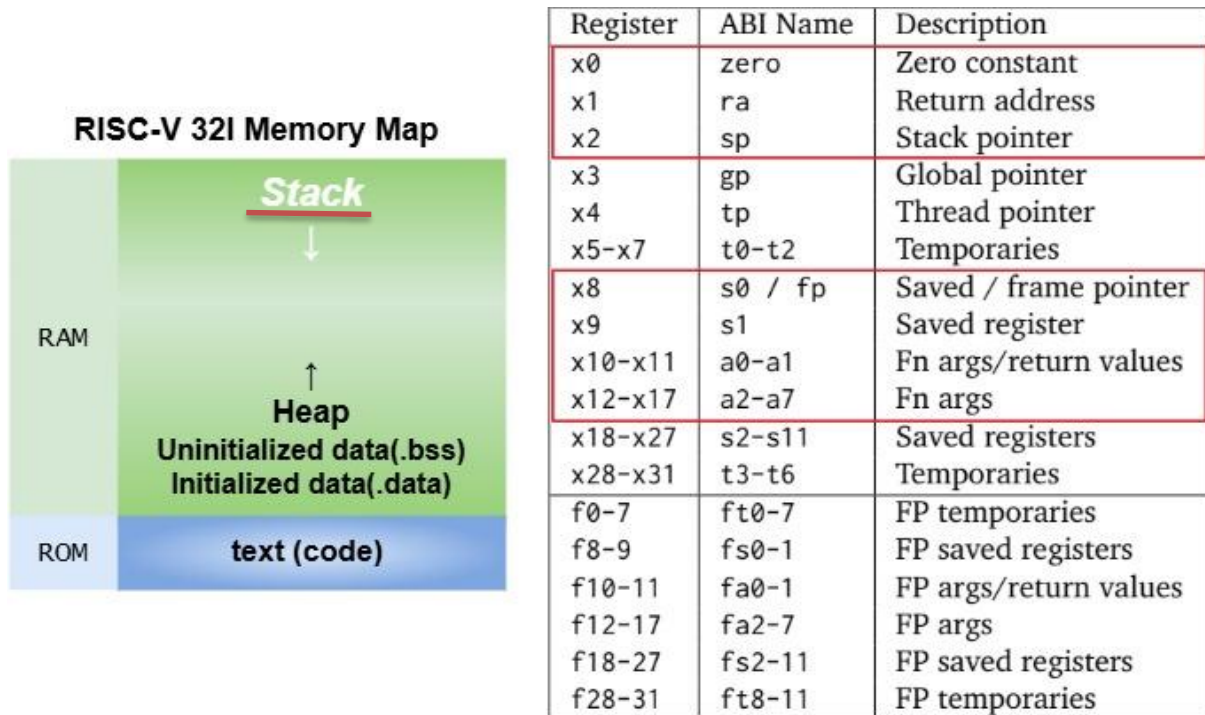
f. U-Type (Upper Immediate)

- **핵심 역할:** 32비트 구조에서 상위 20비트 대형 상수를 다루기 위한 타입으로, 대규모 메모리 주소 접근이나 큰 상수 로드 시에 사용된다.
- **구조적 특징:**
 - LUI: 상위 20비트 Immediate 값을 레지스터 rd의 상위 비트에 채우고, 하위비트는 0으로 설정한다.
 - AUIPC: 상위 20비트 Immediate 값을 현재 PC 값에 더하여 레지스터 rd에 저장하여, PC 상대 주소 계산에 사용된다.

U-TYPE		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
		imm																				rd			opcode								
LUI	rd = imm	imm[31:12]																				rd			0	1	1	0	1	1	1		
AUIPC	rd = PC + imm	imm[31:12]																				rd			0	0	1	0	1	1	1		

3.1.2 Memory Map & Application Binary Interface(ABI)

해당 ISA 규격의 머신 코드가 시스템에서 동작하기 위해선, 메모리 공간의 효율적인 분할과 함수 호출 및 데이터 처리를 위한 레지스터 간의 약속(ABI)가 필수적이다. 아래 이미지는 RISC-V 32I의 기본적인 메모리 맵 구조와 ABI에 따른 범용 레지스터의 역할을 보여준다.



a. Memory Map 구조

- ROM 영역
 - 컴파일된 실제 실행코드가 저장되는 비휘발성 메모리 영역으로, Read-Only로 접근하여 명령어 인출(Instruction Fetch) 단계가 이루어진다.
- RAM 영역
 - Initialized data(.data): 초기값이 있는 전역 변수(Global)나 정적 변수(Static)가 저장된다.
 - Uninitialized data(.bss): 초기값이 없는 전역 변수가 저장되며, 프로그램 시작 시 0으로 자동 초기화된다.
 - Heap: 동적 메모리 할당을 위한 공간으로, 메모리 주소가 낮은 곳에서 높은 곳으로 커지며 확장된다.
 - Stack: 함수 호출 시 필요한 지역 변수, 매개변수, 복귀 주소(Return Address) 등이 저장되는 공간으로, 메모리 주소가 높은 곳에서 낮은 곳으로 감소하며 확장된다.

b. ABI 레지스터 맵 (Register Mapping)

ABI 테이블은 하드웨어의 범용 레지스터(x0-x31)가 Compile 시에 사용될 목적과 명칭을 규정하고 있다. 붉은색 박스의 경우 함수 호출 프로세스, 포인터 제어 등 Stack 할당에 사용되는 핵심 레지스터들로서 아래 설명과 같다.

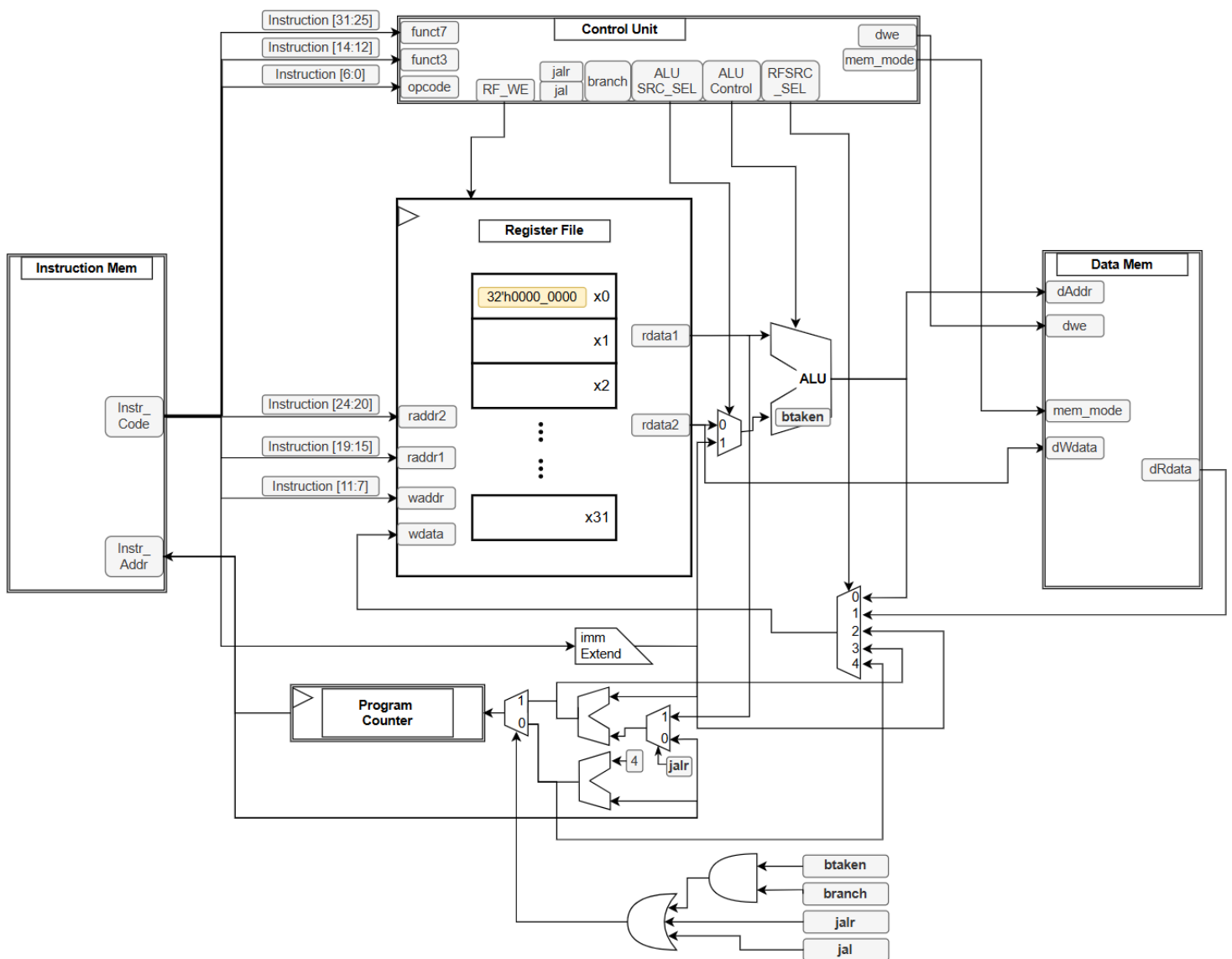
- x0 (zero): 항상 0값을 유지하는 레지스터로, 값을 초기화하거나 조건 비교 시 기준으로 빈번히 활용된다.
- x1 (ra): 함수 호출 시 돌아올 복귀 주소(return address)를 저장한다.
- x2 (sp): 좌측 메모리 맵의 Stack 영역 최하단을 가르키는 Stack Pointer 역할을 수행한다.
- x8 (s0/fp): 함수 프레임의 시작 주소를 기록하는 Frame Pointer 또는 필요에 따라 Saved Register로 쓰인다.
- x9 (s1): 함수 호출 시 값이 보존되어야 하는 데이터를 저장한다.
- x10-x11 (a0-a1): 함수 호출 시 매개변수를 전달함과 동시에, 함수가 종료될 때 결과물(Return Values)을 담아 반환하는 이중 역할을 수행한다.
- x12-x17 (a2-a7): 추가적인 함수 매개변수들을 순차적으로 넘겨주기 위해 사용된다.

4. RISC-V 32I 아키텍처(Architecture)

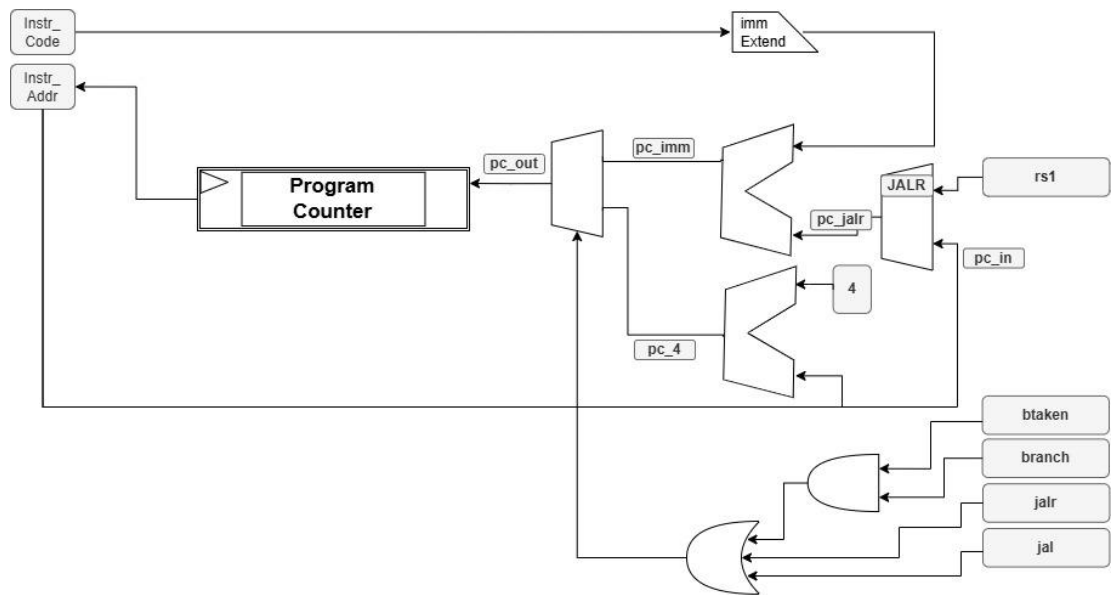
4.1 Microarchitecture 설계

4.1.1 RISC-V CPU Block Diagram

본 설계에서는 앞서 정의된 ISA, ABI 및 Memory Mapping 규격을 기반으로, 컴파일된 머신 코드가 하드웨어 상에서 Single-Cycle내에 오차없이 Fetch-Decode-Execute-Memory Access-Write Back 단계를 수행할 수 있도록 RISC-V CPU Microarchitecture를 설계하였다.

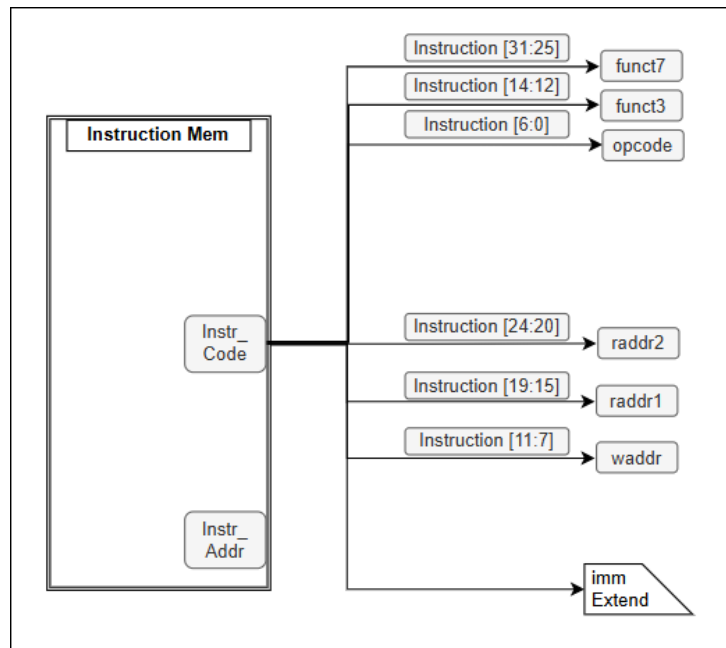


a. Program Counter



- 역할: 현재 실행 중인 명령어의 주소를 보관하고, 다음 사이클에 실행할 명령어 주소 (Instr_Addr)를 생성한다.
- 특징: 기본적으로 현재 PC 값에 4를 더해 다음 주소 PC+4로 이행하지만, 아래쪽 분기 제어 로직((branch&b_taken), jalr, jal)에 의해서 MUX가 전환되면 Extend 블록에서 연산된 Immediate 값이나 레지스터 rs1 주소를 반영하여 분기 타겟 주소로 점프한다.

b. Instruction Memory 및 Imm Extend



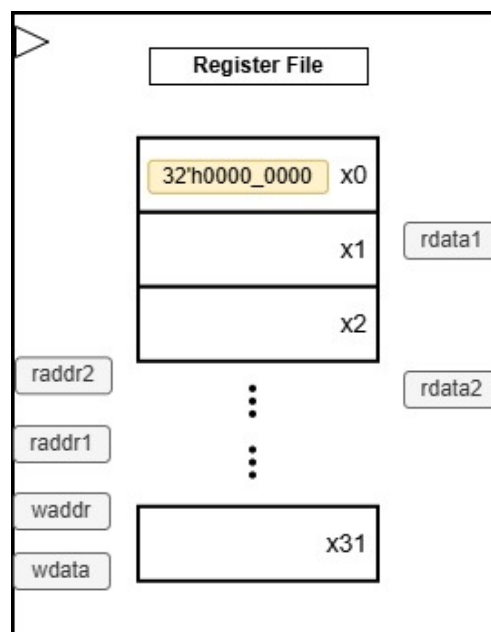
- Instruction Memory
 - 역할 및 특징: 프로그램의 실행코드가 상주하는 읽기 전용 메모리 영역으로 32bit 명령어 코드를 한 사이클마다 인출(Fetch)하여 시스템에 공급한다.
- Imm Extend
 - 역할 및 특징: 명령어 포맷(I, S, B, U, J-Type)에 따라 즉시값(Immediate) 비트들을 하드웨어적으로 정렬하고, 32비트 크기로 부호 확장(Sign Extension)을 수행한다.

c. Control Unit



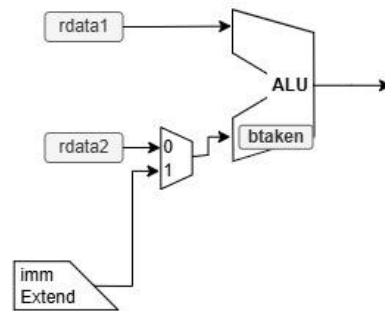
- 역할: 입력된 명령어 비트 중 일부 (funct7, funct3, opcode)를 받아 디코딩한 뒤, 전체 하드웨어 데이터패스를 제어하는 신호들을 생성한다.
- 제어 신호:
 - 레지스터 파일 쓰기 활성화(RF_WE), 점프/분기 제어 신호(jalr, jal, branch)
 - ALU 소스 (rdata2 vs imm) 선택 MUX 신호(ALU_SRC_SEL), ALU 연산 지정 신호(ALU_Control)
 - 데이터 메모리 쓰기 활성화 및 읽기 모드 신호(dwe, mem_mode)
 - 최종 Writeback 데이터를 선택하는 멀티플렉서 제어 신호(RFSRC_SEL)

d. Register File



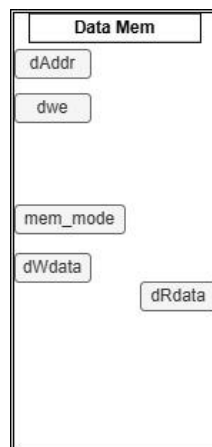
- 역할: RISC-V ABI 규격에 따른 32개의 범용 레지스터 (x0~x31) 집합체이다.
- 특징: 명령어의 고정된 비트 필드 위치에서 raddr1 (Instruction[19:15])과 raddr2 (Instruction[24:20]) 주소를 받아 두 개의 소스 데이터 (rdata1, rdata2)를 출력한다. 명령어 타입별 연산 결과는 필요시 waddr (Instruction[11:7]) 주소의 레지스터에 저장되며, 레지스터 x0는 항상 32'h0000_0000으로 고정된다.

e. ALU



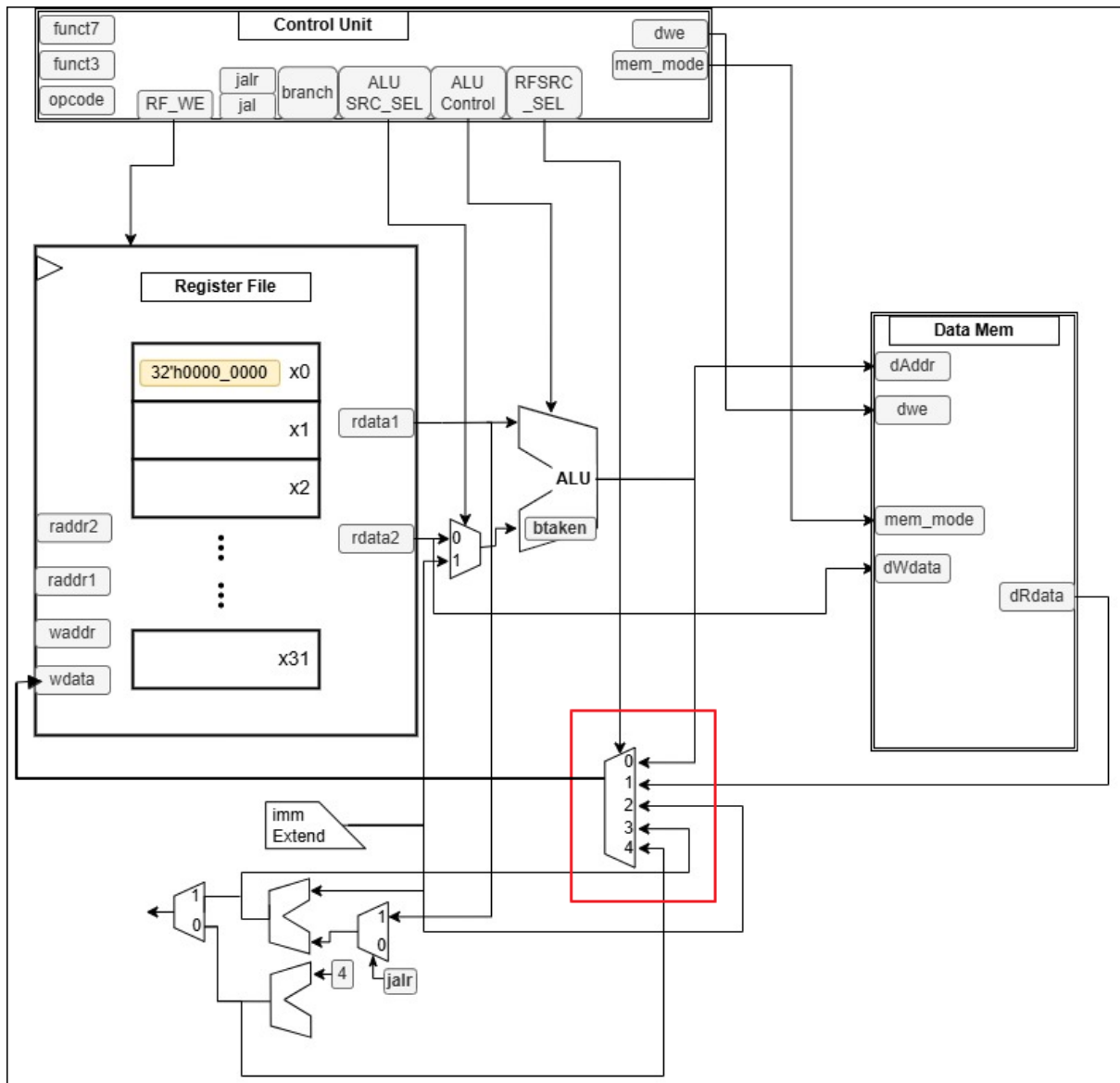
- 역할: 프로세서의 실질적인 연산 핵심부로, 산술 (ADD, SUB), 논리 (AND, OR, XOR), 시프트 (SLL, SRL, SRA) 연산을 수행한다.
- 특징: 첫 번째 입력은 항상 **rdata1** 고정이지만, 두 번째 입력은 **ALU_SRC_SEL** 신호에 의해 제어되는 MUX를 통과하여 레지스터 출력값 (**rdata2**) 혹은 즉시값 확장 데이터 (**Imm Extend**) 중 하나를 선택적으로 받아 연산한다. B-Type 명령어에 한해 분기 명령어 비교를 위한 **btaken** 플래그를 출력한다.

f. Data Memory



- 역할: 프로그램의 데이터 영역 (.data, .bss, Stack, Heap)을 담당하는 실질적인 데이터 저장소이다.
- 특징: ALU가 계산한 결과 주소 (**dAddr**)를 기반으로 작동한다. Store 명령 시 **dwe** 신호가 활성화되어 레지스터에서 읽어온 **dWdata** (**rdata2** 라인과 연결됨)를 메모리에 기록하고, Load 명령 시 메모리에서 **dRdata**를 읽어 출력한다. 또한, **mem_mode** 신호를 통해 Byte, Half-word, Word 단위 접근을 제어한다.

g. WriteBack 5-to-1 MUX



- 역할 및 특징: 레지스터 파일 입력값 (`wdata`)으로 들어갈 최종값을 결정하는 MUX로, `RFSRC_SEL` 신호에 따라 [①] ALU 연산 결과, [②] 데이터 메모리 로드 값 (`dRdata`), [③] `PC + 4` (함수 호출 리턴 주소), [④ or ⑤] 루프나 대형 상수 처리를 위한 주소 연산 값 등을 선택하여 레지스터 파일로 최종 Writeback 시킨다.

4.2 Single-Cycle 프로세서

본 설계에서 구현한 RISC-V CPU는 단일 사이클(Single-Cycle) 마이크로아키텍처를 기반으로 동작한다. 이는 하나의 클록 주기(One Clock Cycle) 내에 하나의 명령어를 완전히 처리하는 방식이다.

모든 명령어는 단일 클록 에지 사이에서 **Fetch**(인출) → **Decode**(해독) → **Execute**(실행) → **Memory Access**(메모리 접근) → **Writeback**(결과 저장)의 표준 5단계를 거치며 연산을 완료한다.

※ 명령어 타입별 Instruction Bit Mapping

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
funct7							rs2				rs1				funct3				rd				opcode							R		
imm[12,10:5]							rs2				rs1				funct3				imm[4:1,11]				opcode							B		
imm[11:5]							rs2				rs1				funct3				imm[4:0]				opcode							S		
imm[11:0]											rs1				funct3				rd				opcode							I		
imm[11:0]											rs1				funct3				rd				opcode							IL		
imm[11:0]											rs1				funct3				rd				opcode							JL		
imm[20,10:1,11,19:12]												rd										opcode							J			
imm[31:12]												rd										opcode							U			

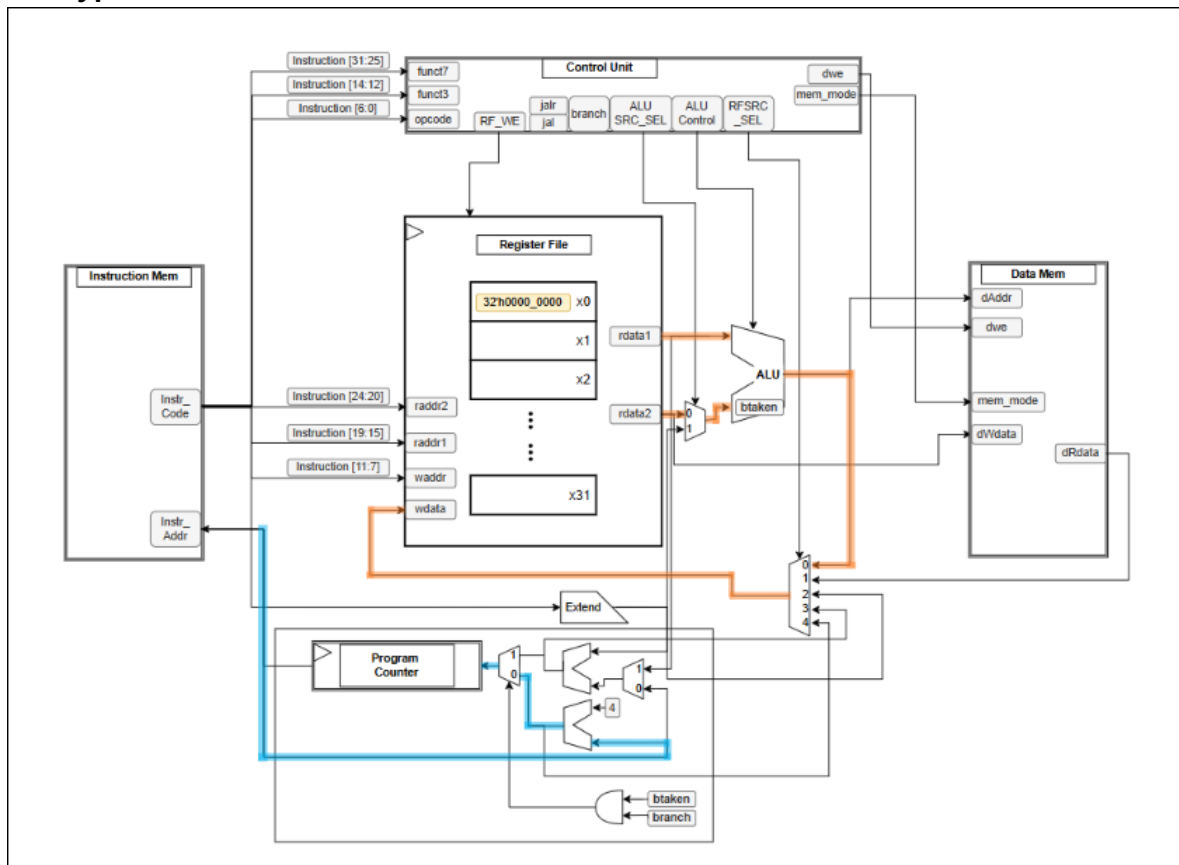
Opcode에 따른 타입별 명령어를 정리한 아래 표를 살펴보면, funct7 및 funct3에 따른 Control Signal 출력으로 Data Flow가 변화하며 원하는 연산을 완료하는 것을 확인할 수 있다.

※ Control Signal 정리 Table

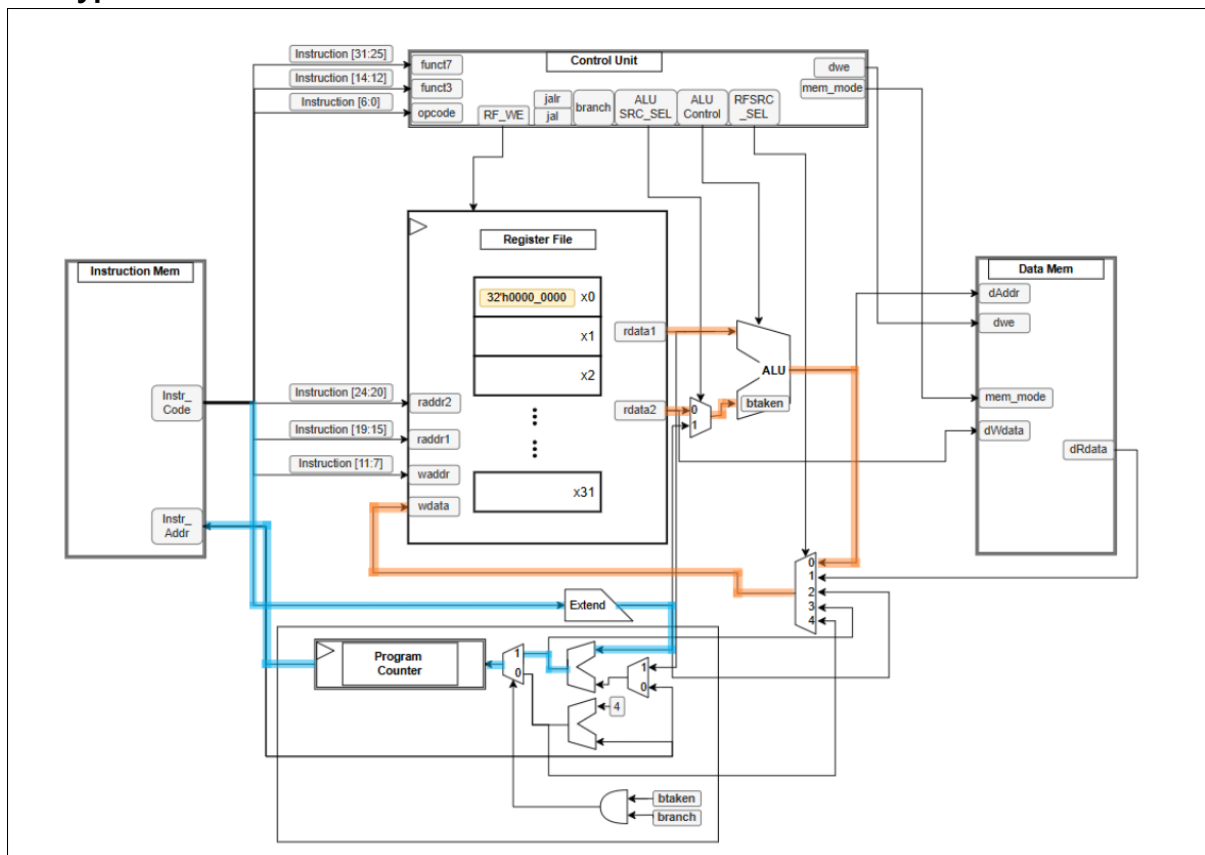
Type	RF_WE	ALUSRC_SEL	ALU_Control	RFSRC_SEL	mem_mode	dwe	branch	jal	jalr
R	1	0	{funct7[5], funct3}	3'b000	3'b000	0	0	0	0
B	0	0	{1'b0, funct3}	3'b000	3'b000	0	1	0	0
S	0	1	`ADD	3'b000	funct3	1	0	0	0
I	1	1	{funct7[5], funct3} or {1'b0, funct3}	3'b000	funct3	0	0	0	0
IL	1	1	`ADD	3'b001	funct3	0	0	0	0
JL	1	0	0	3'b100	3'b000	0	0	0	1
J	1	0	0	3'b100	3'b000	0	0	0	0
U	1	0	0	LUI = 3'b010 AIUPC = 3'b011	0	0	0	0	0

위의 Instruction Bit Mapping 및 Control Signal 정리표를 참고하여, Data 및 PC의 연산 Flow를 타입별로 살펴보았다.

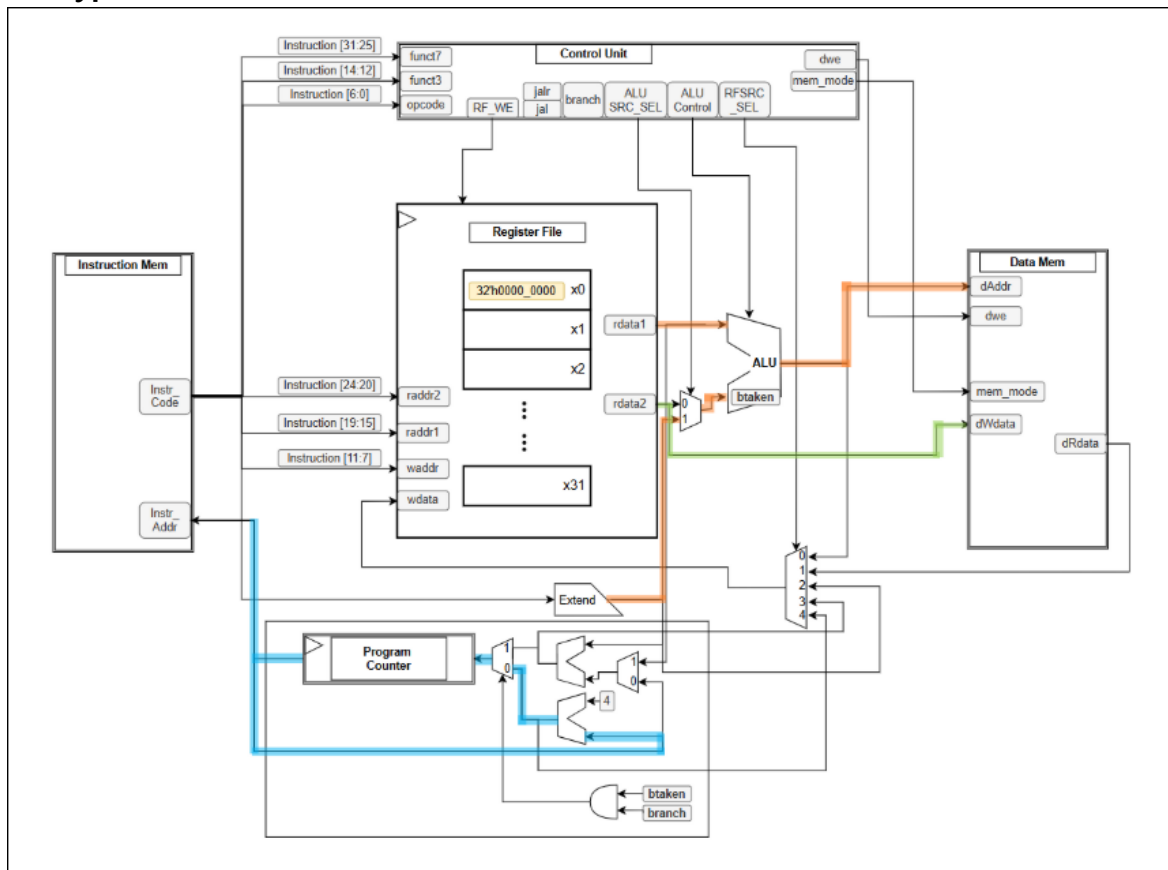
- R-Type



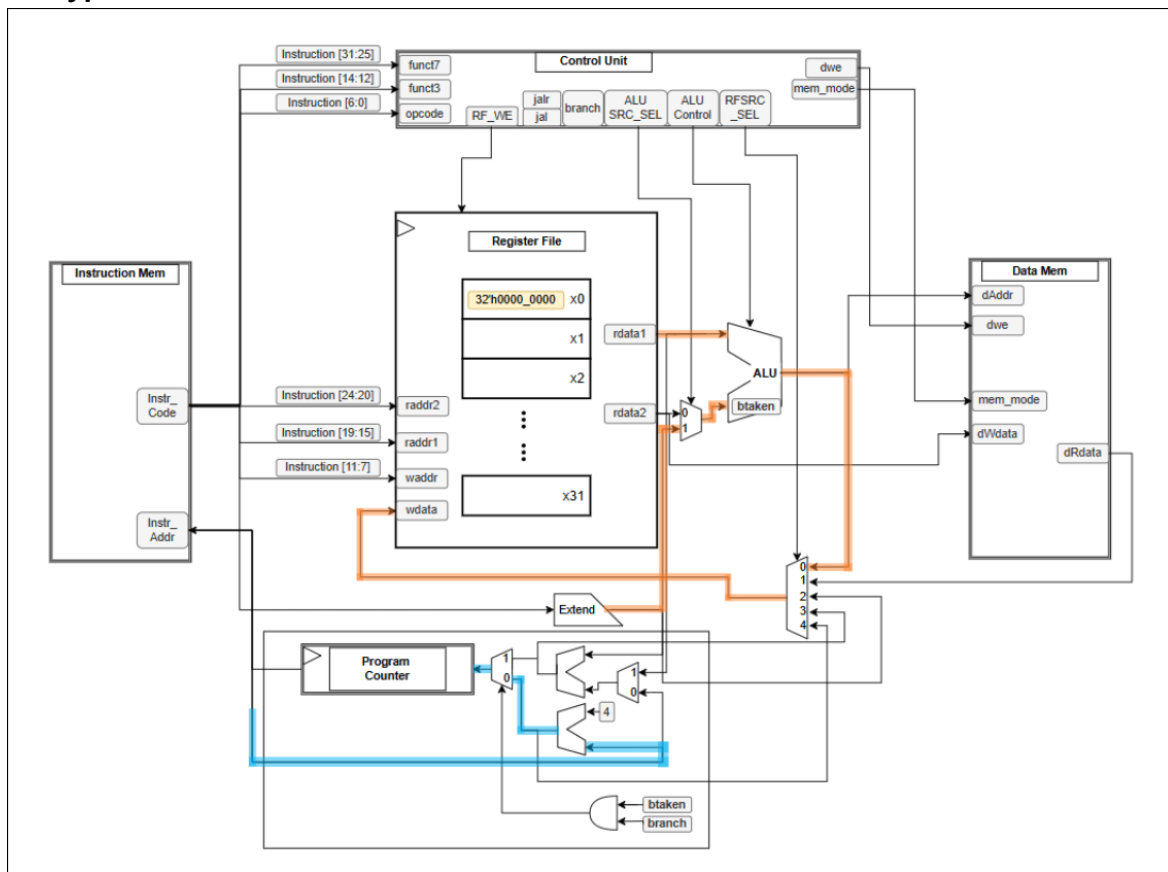
- B-Type



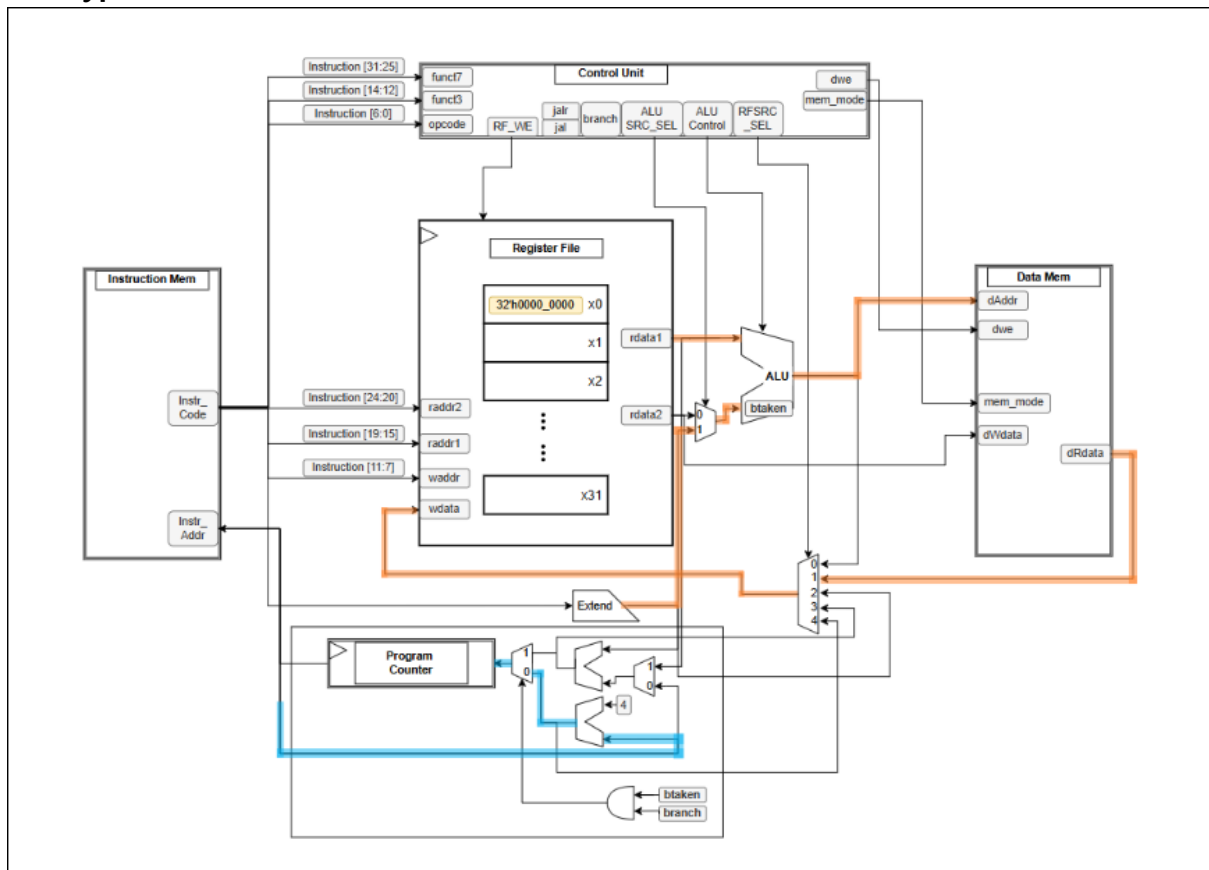
- S-Type



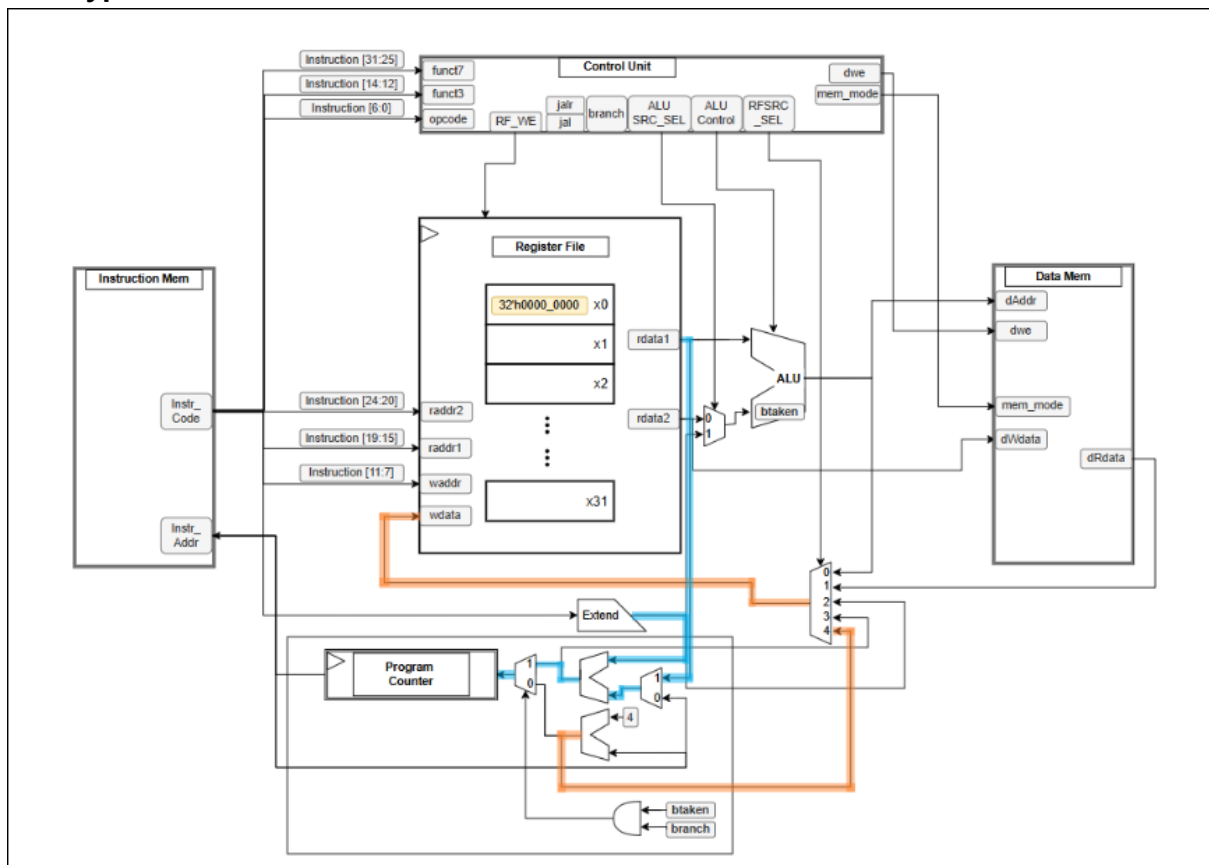
- I-Type



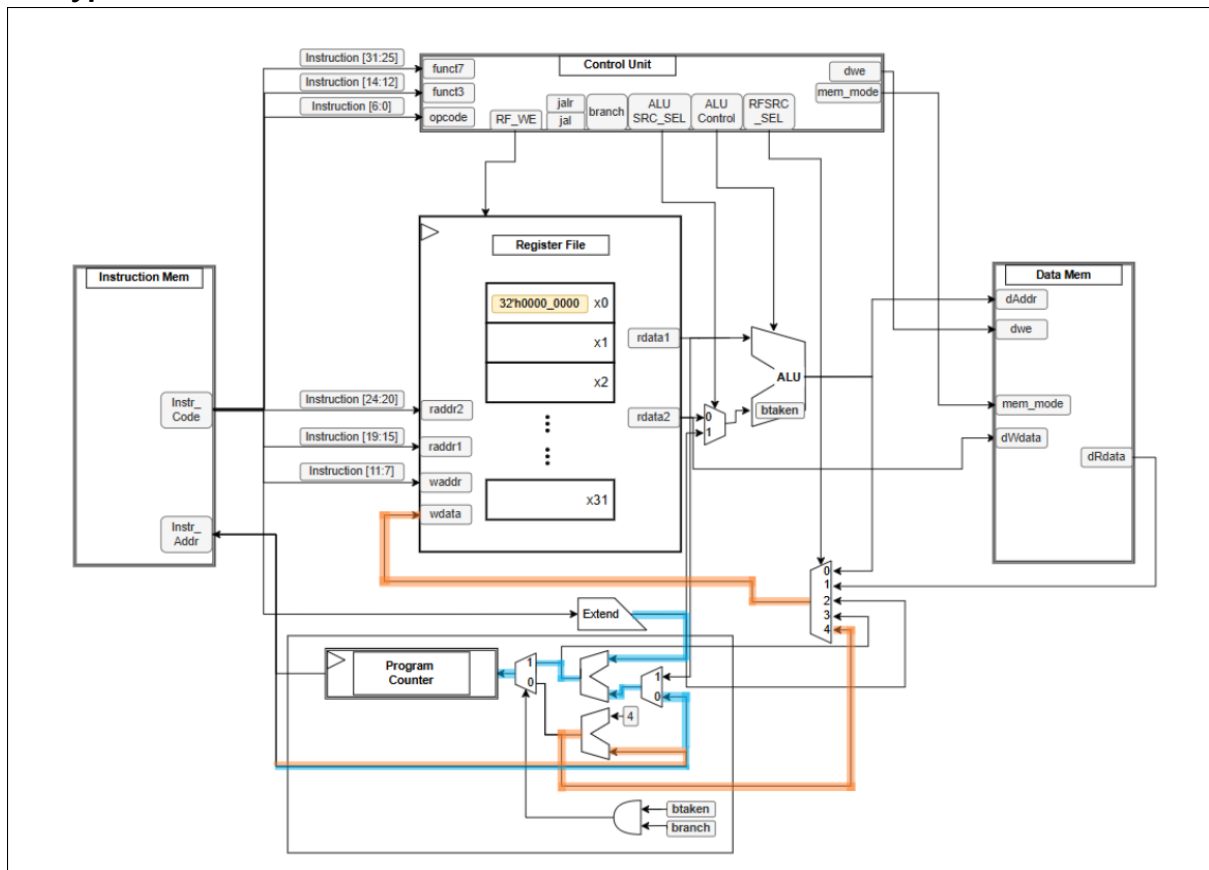
- IL-Type



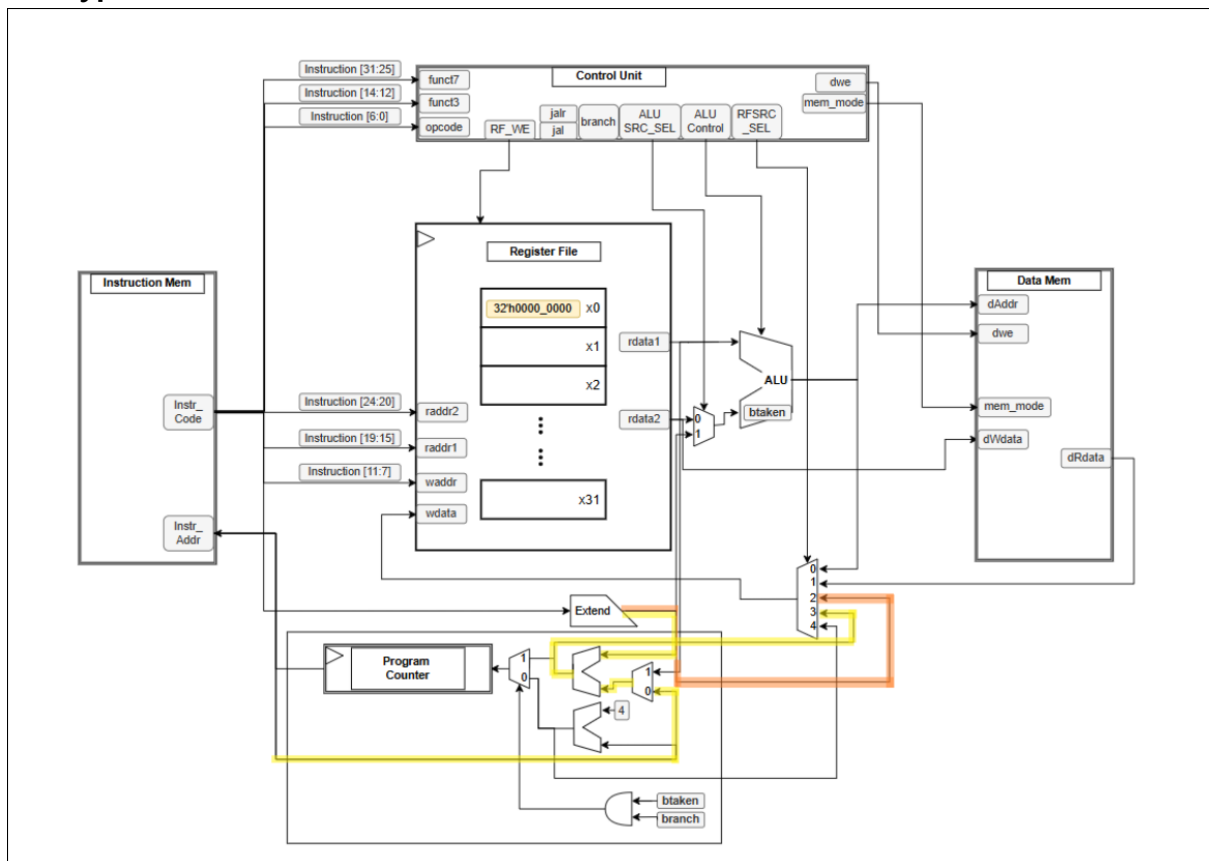
- JL-Type



- J-Type



- U-Type



5. C 코드 검증 시나리오

5.1 Bubble Sort

설계한 RISC-V 32I Single-Cycle 프로세서의 명령어 처리 성능과 제어 흐름 제어 능력을 검증하기 위해, 중첩 루프 및 다중 함수 호출 구조를 가진 버블 정렬 알고리즘을 수행하여 메모리와 레지스터 파일의 하드웨어적 변화를 추적 분석하고자 하였다. 인접한 두 원소의 위치를 교환하는 버블 정렬은 구조상 아래 그림 (Code for Bubble Sort)와 같이 main, sort, swap의 3단계 계층적 함수 호출을 동반한다.

Code for Bubble Sort

```
void sort(int *pNum, int size);
void swap(int *a, int *b);
void main(void){
    int Num[6] = {3,5,9,1,7};
    int a = 0;
    sort(Num,5);

    a=0x12345678;

    while(1);
    return;
}

void sort(int *pNum, int size){
    for(int i=0; i<size; i++){
        for(int j=0; j<size-i; j++){
            if(pNum[j] > pNum[j+1]){
                swap(&pNum[j], &pNum[j+1]);
            }
        }
    }
}

void swap(int *a, int *b){
    int temp = *a;
    *a = *b;
    *b = temp;
    return;
}
```

Machine Language / Assembly Language

```
0000000000000000
:
0: fd010113      addi  sp,sp,-48
4: 02112623      sw   ra,44(sp)
8: 02812423      sw   s0,40(sp)
c: 03010413      addi  s0,sp,48
10: fc042a23      sw   zero,-44(s0)
14: fc042c23      sw   zero,-40(s0)
18: fc042e23      sw   zero,-36(s0)
1c: fe042023      sw   zero,-32(s0)
20: fe042223      sw   zero,-28(s0)
24: fe042423      sw   zero,-24(s0)
28: 00300793      li   a5,3
2c: fcf42a23      sw   a5,-44(s0)
30: 00500793      li   a5,5
34: fcf42c23      sw   a5,-40(s0)
38: 00900793      li   a5,9
3c: fcf42e23      sw   a5,-36(s0)
40: 00100793      li   a5,1
44: fef42023      sw   a5,-32(s0)
48: 00700793      li   a5,7
4c: fef42223      sw   a5,-28(s0)
50: fe042623      sw   zero,-20(s0)
54: fd440793      addi  a5,s0,-44
58: 00500593      li   a1,5
5c: 00078513      mv   a0,a5
60: 014000ef      jal  ra,74
64: 123457b7      lui  a5,0x12345
68: 67878793      addi  a5,a5,1656
```

또한, 우측 이미지는 C 코드(High-level Language)를 Compiler → Assembler → Linker를 통해 Machine Language 및 Assembly Language로 변환한 이미지다. 설계한 CPU에서 사용자가 원하는 동작을 오류없이 수행하는 것을 Machine Language 및 Waveform 비교 분석을 통해 살펴보고자 한다.

5.1.1 main 함수

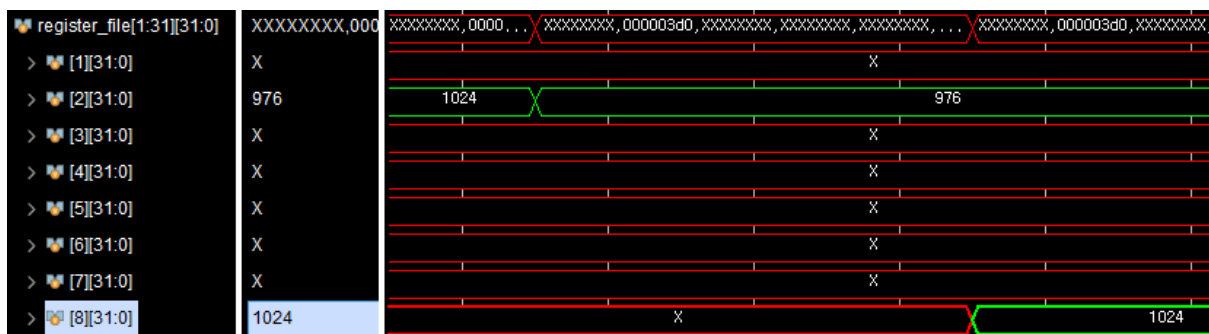
즉 main 함수 분석을 통해 레지스터 및 메모리 레벨에서 어떻게 변환되고 실행되는지, 즉 지역 변수 및 배열의 Stack Mapping을 확인 및 검증해보았다.

1. 스택 프레임 생성 및 포인터 설정

프로세서는 `addi sp, sp, -48`을 통해 스택 포인터를 아래로 내려 48byte의 스택 공간을 확보하고, 현재 프레임의 기준 주소 역할을 할 Frame Pointer(`s0`)를 `s0 = sp + 48`로 설정하여 기준점을 잡는다.

```
0: fd010113      addi sp, sp, -48
4: 02112623      sw  ra, 44(sp)
8: 02812423      sw  s0, 40(sp)
c: 03010413      addi s0, sp, 48
```

● Register File x2, x8(sp, s0) Waveform



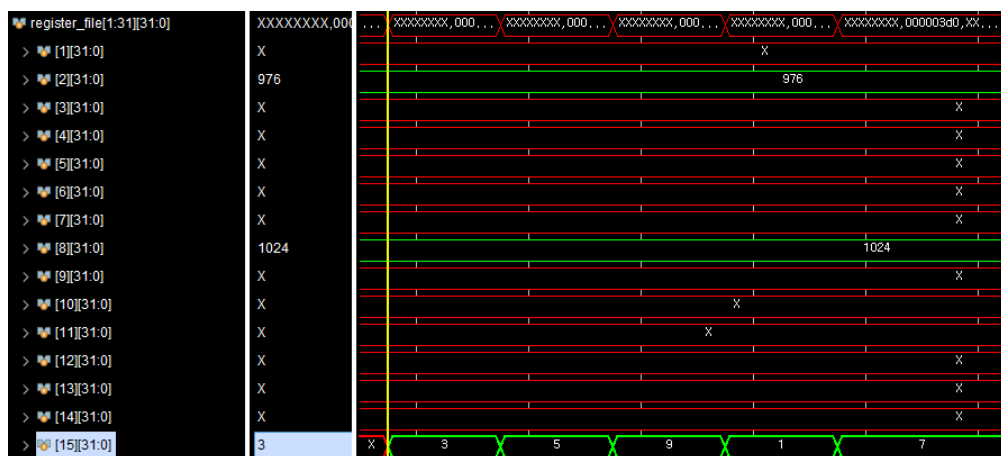
2. 배열 Num[6]의 연속적 메모리 할당

main 영역에 선언된 정수형 배열 데이터(3, 5, 9, 1, 7)는 s0 기준 오프셋 주소(-44(s0), -40(s0), -36(s0) 등)에 sw(Store Word) 명령어를 연속적으로 수행함으로써 데이터 메모리의 RAM Stack 영역에 탑재된다.

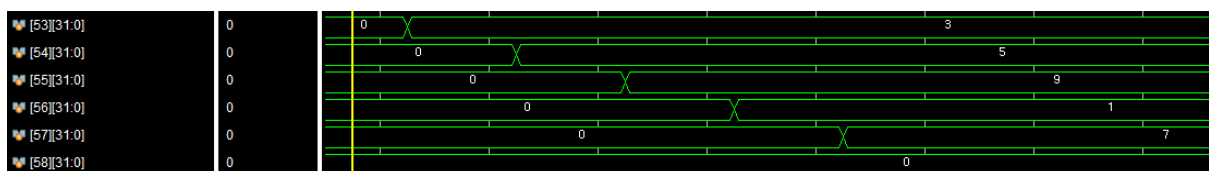
```
28: 00300793    li    a5, 3
2c: fcf42a23    sw    a5, -44(s0)    # Num[0] = 3
30: 00500793    li    a5, 5
34: fcf42c23    sw    a5, -40(s0)    # Num[1] = 5
38: 00900793    li    a5, 9
3c: fcf42e23    sw    a5, -36(s0)    # Num[2] = 9
...

```

● Register File x15(a5) Waveform



● Data Memory Waveform



3. 32비트 대형 상수 대입 (a=0x12345678)

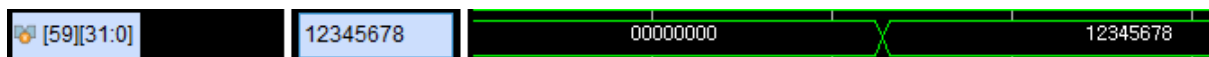
```
64: 123457b7    lui  a5, 0x12345
68: 67878793    addi a5, a5, 1656  # 1656 = 0x678
6c: fef42623    sw   a5, -20(s0)
```

RISC-V 명령어는 32비트 고정 길이로 하나의 명령어로 32비트 상수 0x12345678을 한번에 로드할 수 없다. 이에 상위 20비트를 로드하는 lui와 하위 12비트를 더하는 addi 명령어가 쌍으로 조합되어 -20(s0)에 sort 함수 종료 후에 최종 저장되는 것을 볼 수 있다.

- Register File x15(a5) Waveform



- Data Memory Waveform



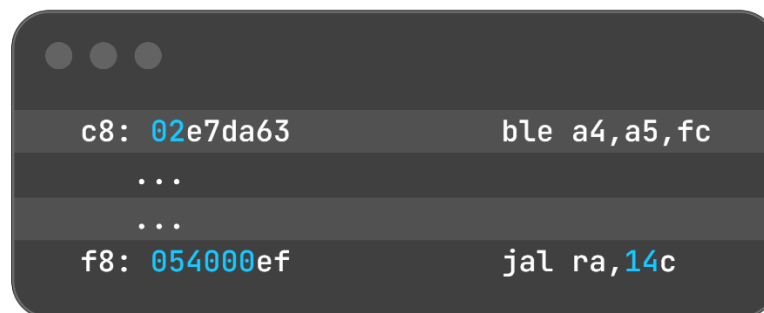
5.1.2 sort 함수

sort 함수는 중첩 루프(Outer i loop, Inner j loop)를 구동하여 데이터의 대소 관계를 비교하고 제어 흐름 분기를 처리하기 위해 다양한 하드웨어 분기 명령어를 사용한다.

1. 조건문(IF) 제어를 위한 분기 메커니즘 분석

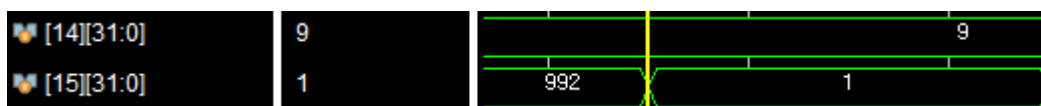
코드 내부의 `if(pNum[j] > pNum[j+1])` 비교 연산은 `ble(Branch if Less than or Equal)` 명령어로 변환된다. `a4` (`pNum[j]`) 값이 `a5` (`pNum[j+1]`) 값보다 작거나 같으면(조건 미성립), 값의 위치를 바꿀 필요가 없으므로 `swap` 호출부를 건너 뛰고 다음 인덱스 연산 주소인 `fc`번지로 제어 흐름을 분기(Jump) 시킨다.

반대로 `a4`가 `a5`보다 크면(조건 성립), `ble` 명령어의 분기가 성립하지 않아 프로그램 카운터(PC)는 자연스럽게 다음 아래 줄로 진행해 `f8: jal ra, 14c` 명령어를 만나 `swap` 함수를 호출하게 되는 구조이다.



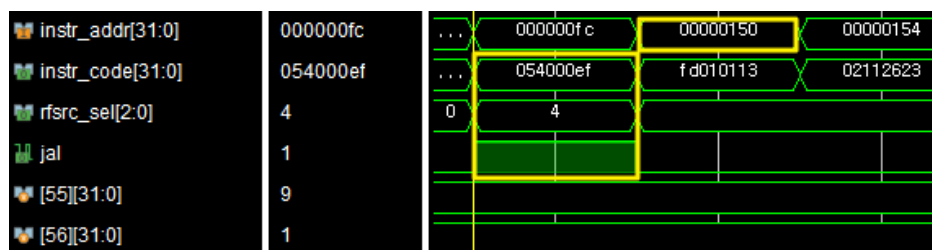
- Register File x15(a5) Waveform

c8: `a4` (9)와 `a5` (1)을 비교하여 조건이 성립해 다음 줄로 진행하게 된다.



- Data Memory Waveform

f8: 명령어 `054000ef`를 만나 `swap`함수 주소(`14c`)로 Jump하게 된다.



5.1.3 swap 함수 분석을 통한 포인터 역참조 및 데이터 교환

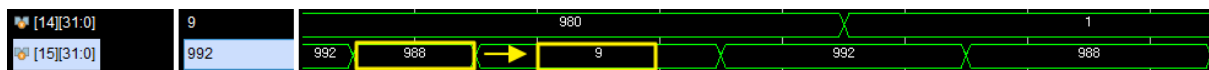
swap 함수는 전달받은 주소(포인터)를 활용하여 하드웨어 메모리 간접 참조를 수행하고, 정렬 조건에 위배되는 두 데이터의 실제 하드웨어 저장 위치를 상호 교환하는 핵심 메커니즘을 보여준다.

1. 포인터 수신 및 주소 기반 데이터 로드(lw)

main 및 sort 함수에서 넘겨준 인자는 배열의 값이 아닌 '메모리 주소'이며, 이는 a0, a1 레지스터를 통해 전달됩니다. 또한, 168번지의 lw a5, 0(a5) 명령어는 레지스터 내부의 데이터를 순수 상수가 아닌 메모리 주소(Address)로 매핑하여 해당 칸의 실제 알맹이를 읽어오는 포인터 역참조(Dereference)의 구체적인 하드웨어적 실행 과정을 보여준다.

```
15c: fca42e23    sw    a0, -36(s0) # 매개 변수 a(&Num[j] 주소)를 스택에 보관
160: fcb42c23    sw    a1, -40(s0) # 매개 변수 b(&Num[j+1] 주소)를 스택에 보관
...
164: fdc42783    lw    a5, -36(s0) # 스택에서 포인터 a가 가리키는 '실제 배열 주소'를 a5
로 로드
168: 0007a783    lw    a5, 0(a5)   # 포인터 역참조: a5 주소지에 위치한 실제 데이터(*a)
를 로드
16c: fef42623    sw    a5, -20(s0) # 임시 변수 t에 복사 (t = *a)
```

● Register File x15(a5) Wave



2. 메모리 직접 갱신을 통한 데이터 교차 저장 (sw)

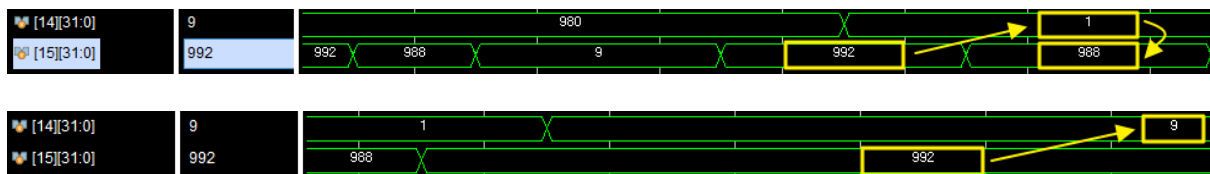
sw a4, 0(a5) 명령어는 교차된 정렬 데이터를 원래 포인터가 가리키고 있던 원본 배열의 스택 주소 공간에 직접 덮어쓰게 된다. 이 과정은 단순한 값의 전달(Call-by-Value)과 달리, 주소 전달 방식(Call-by-Reference)이 하드웨어 레벨에서 lw(Load)와 sw(Store)의 유기적인 결합을 통해 원본 배열 데이터를 영구적으로 수정해 나가는 흐름을 보여준다.

```

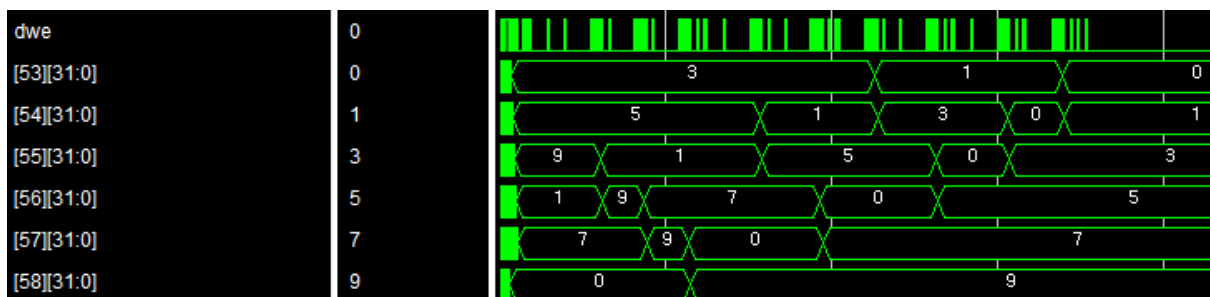
170: fd842783    lw    a5, -40(s0) # 포인터 b의 주소를 a5로 로드
174: 0007a703    lw    a4, 0(a5)   # a4 = *b (b가 가리키는 메모리의 실제 데이터)
178: fdc42783    lw    a5, -36(s0) # 다시 포인터 a의 주소를 a5로 로드
17c: 00e7a023    sw    a4, 0(a5)   # 포인터 a의 주소지에 데이터 *b를 저장 (*a = *b)

180: fd842783    lw    a5, -40(s0) # 포인터 b의 주소를 a5로 로드
184: fec42703    lw    a4, -20(s0) # 임시 변수 t의 값(*a였던 것)을 a4로 로드
188: 00e7a023    sw    a4, 0(a5)   # 포인터 b의 주소지에 t의 값을 저장 (*b = t)
  
```

● Register File x14, x15(a4, a5) Waveform



● Data Memory Waveform (전체)

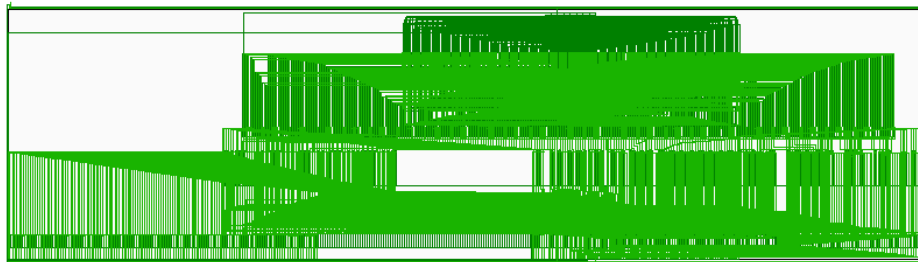


5. Trouble-shooting

5.1 Data Memory

5.1.1 Issue 소개

Data Memory 설계 시, Vivado가 메모리로 인지하지 못하고, 아래 그림과 같이 수많은 레지스터 및 MUX로 합성하는 현상을 발견했다.



5.1.2 Root Cause 및 트러블 슈팅

수차례 코드를 변경해보았으나 똑같이 합성기가 메모리로 인지하지 못했고, 이에 기존 Multi-Dimension으로 작성한 Address Decoding 코드를 1차원 배열로 수정 후 합성기가 메모리로 인식해 회로를 만들어주는 것을 확인하였다.

이에 XILINX 사에서 배포한 U901 - Vivado Design Suite User Guide 를 확인해본 결과, Chapter 7 멀티디멘션 배열 사용에 대한 아래 주의사항을 찾을 수 있었고, 다차원 배열 사용 시 단일 요소 선택 제약, 이에 복잡한 인덱싱 적용 시 메모리 구성이 되지 않음을 확인할 수 있었다.

Multi-Dimensional Arrays

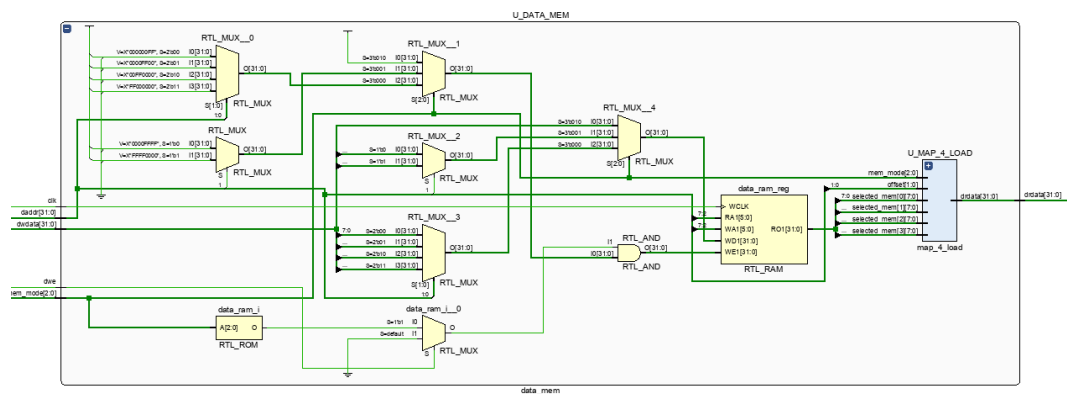
Vivado synthesis supports multi-dimensional array types of up to two dimensions.

- Multi-dimensional arrays can be:
 - Any net
 - Any variable data type
- Code assignments and arithmetic operations with arrays.
- You cannot select more than one element of an array at one time.
- You cannot pass multi-dimensional arrays to:
 - System tasks or functions
 - Regular tasks or functions

Before	After
<pre> logic [3:0][7:0] data_ram[0:63]; // logic [1:0] offset; logic [5:0] ram_id; assign offset = daddr[1:0]; assign ram_id = daddr[7:2]; always_ff @(posedge clk) begin if(dwe) begin case(mem_mode) `SW: begin                 data_ram[ram_id] <= dwdata;             end             `SH: begin case(offset[1]) 1'd0: data_ram[ram_id][1:0] <= dwdata[15:0]; 1'd1: data_ram[ram_id][3:2] <= dwdata[15:0]; endcase end `SB: begin case(offset) 2'd0: data_ram[ram_id][0] <= dwdata[7:0]; 2'd1: data_ram[ram_id][1] <= dwdata[7:0]; 2'd2: data_ram[ram_id][2] <= dwdata[7:0]; 2'd3: data_ram[ram_id][3] <= dwdata[7:0]; endcase end endcase end end end </pre>	<pre> logic [31:0] data_ram[0:63]; // logic [1:0] offset; logic [5:0] ram_id; assign offset = daddr[1:0]; assign ram_id = daddr[7:2]; always_ff @(posedge clk) begin if(dwe) begin case(mem_mode) `SW: begin                 data_ram[ram_id] <= dwdata;             end             `SH: begin case(offset[1]) 1'd0: data_ram[ram_id][15:0] <= dwdata[15:0]; 1'd1: data_ram[ram_id][31:16] <= dwdata[15:0]; endcase end `SB: begin case(offset) 2'd0: data_ram[ram_id][7:0] <= dwdata[7:0]; 2'd1: data_ram[ram_id][15:8] <= dwdata[7:0]; 2'd2: data_ram[ram_id][23:16] <= dwdata[7:0]; 2'd3: data_ram[ram_id][31:24] <= dwdata[7:0]; endcase end endcase end end end </pre>

5.1.3 개선 결과 및 교훈

코드를 1차원 배열로 변경한 후에 아래와 같이 메모리로 인지하는 것을 확인할 수 있었다. 이 트러블슈팅을 통해 Official User Guide를 확인할 수 있는 시간을 가졌고, 여러 상황에서 선택의 기준이 되어주는 매뉴얼의 중요성을 다시 한번 체감할 수 있었다.



5.2 Negative Slack Issue

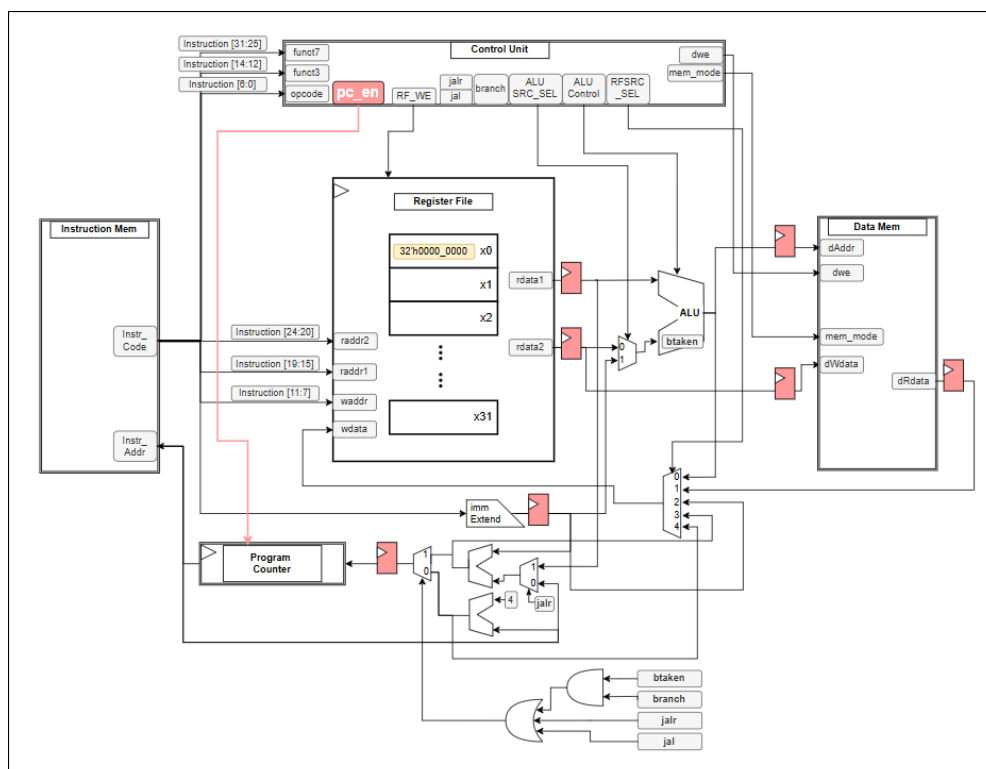
5.2.1 Issue 소개

설계한 Single-Scycle CPU를 Synthesis 및 Implemenation 후에 Timing을 확인한 결과, 아래와 같이 Negative Slack이 -4.051ns가 측정되어 타이밍 마진 부족 현상이 있었다.

Timing	
Worst Negative Slack (WNS):	-4.051 ns
Total Negative Slack (TNS):	-370.734 ns
Number of Failing Endpoints:	280
Total Number of Endpoints:	937
Implemented Timing Report	

5.1.2 Root Cause 및 트러블 슈팅

모든 명령어가 단일 사이클 내에 Fetch – Decode – Execute - Memory Access - Writeback 모두 완료되어야 하는 Single-Cycle 아키텍처 구조적 특성에서 기인한다고 생각하였다. 이에 동작 주파수를 내리기보다는 Multi-Cycle 방식을 도입하여 Critical Path를 물리적으로 분할해보기로 하였다.



Fetch / Decode / Execute / Memory Access / Writeback 각각의 단계에 맞춰 위 그림과 같이 pc_en 신호 및 Register를 추가하여, 복잡도가 낮은 명령어(R-Type, S-Type 등)은 최소 사이클(3~4 Cycles) 내에 조기 완결시키고, 데이터 메모리 참조가 동반되는 명령어(IL-Type, Load)에만 최대 사이클(5 Cycles)을 할당해주었다.

5.1.3 개선 결과 및 교훈

Multi-Cycle로 설계를 변경한 결과 WNS (Worst Negative Slack)가 -4.051ns에서 2.302ns로 개선되는 것을 확인할 수 있었다. 이번 트러블 슈팅을 통해 Negative Slack의 개념을 실무적으로 이해하였으며, 시스템 요구사항에 따라 아키텍처를 최적화하는 하드웨어 설계의 유연성과 Multi-Cycle 구조의 이점을 체득할 수 있었다.

Timing	
Worst Negative Slack (WNS):	2.302 ns
Total Negative Slack (TNS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	1049
Implemented Timing Report	

6. 결론

6.1 결과 요약

- **RISC-V 마이크로아키텍처 구현**
 - SystemVerilog 언어를 활용하여 RISC-V 32I 기반의 Single-Cycle CPU Core를 구현하였다.
- **소프트웨어 알고리즘을 통한 런타임 기능 검증**
 - 컴파일러 툴체인을 거쳐 인코딩된 버블 정렬(Bubble Sort) 기계어 코드를 하드웨어 메모리에 탑재하고 실행함으로써, main-sort-swap으로 이어지는 다중 함수 호출 시의 레지스터 파일 제어 및 데이터 메모리 스택(Stack) 영역 런타임 매핑을 검증하였다.

6.2 배운점 및 고찰

- **하드웨어-소프트웨어 경계 인터페이스(HW/SW Interface) 이해**
 - 고수준 언어(C)로 작성된 알고리즘이 컴파일러 툴체인(Compiler → Assembler → Linker)의 전처리 과정을 거쳐 32비트 고정 길이 명령어 포맷(ISA)인 기계어로 변환되는 과정을 분석하였다. 이를 통해 관념적인 소프트웨어 코드가 레지스터 파일 및 메모리 맵(ABI) 상에서 실제 하드웨어의 물리적 주소(Physical Address)와 1:1로 정렬되어 제어되는 메커니즘을 파악하였다.
- **Multi-Cycle 마이크로아키텍처의 자원 최적화 및 유연성 경험**
 - 모든 연산이 단일 사이클 내에 완료되어야 했던 Single-Cycle 방식의 벗어나, 하나의 명령어를 다수의 세부 클록 주기(State)로 분할 처리하는 방식을 분석하였다. 이를 통해 실제 런타임 시의 평균 실행 사이클 수를 동적으로 최적화하는 제어 기법을 습득할 수 있었다.